



**DIKTISAINTEK
BERDAMPAK**

COMPUTER VISION (TFP505)

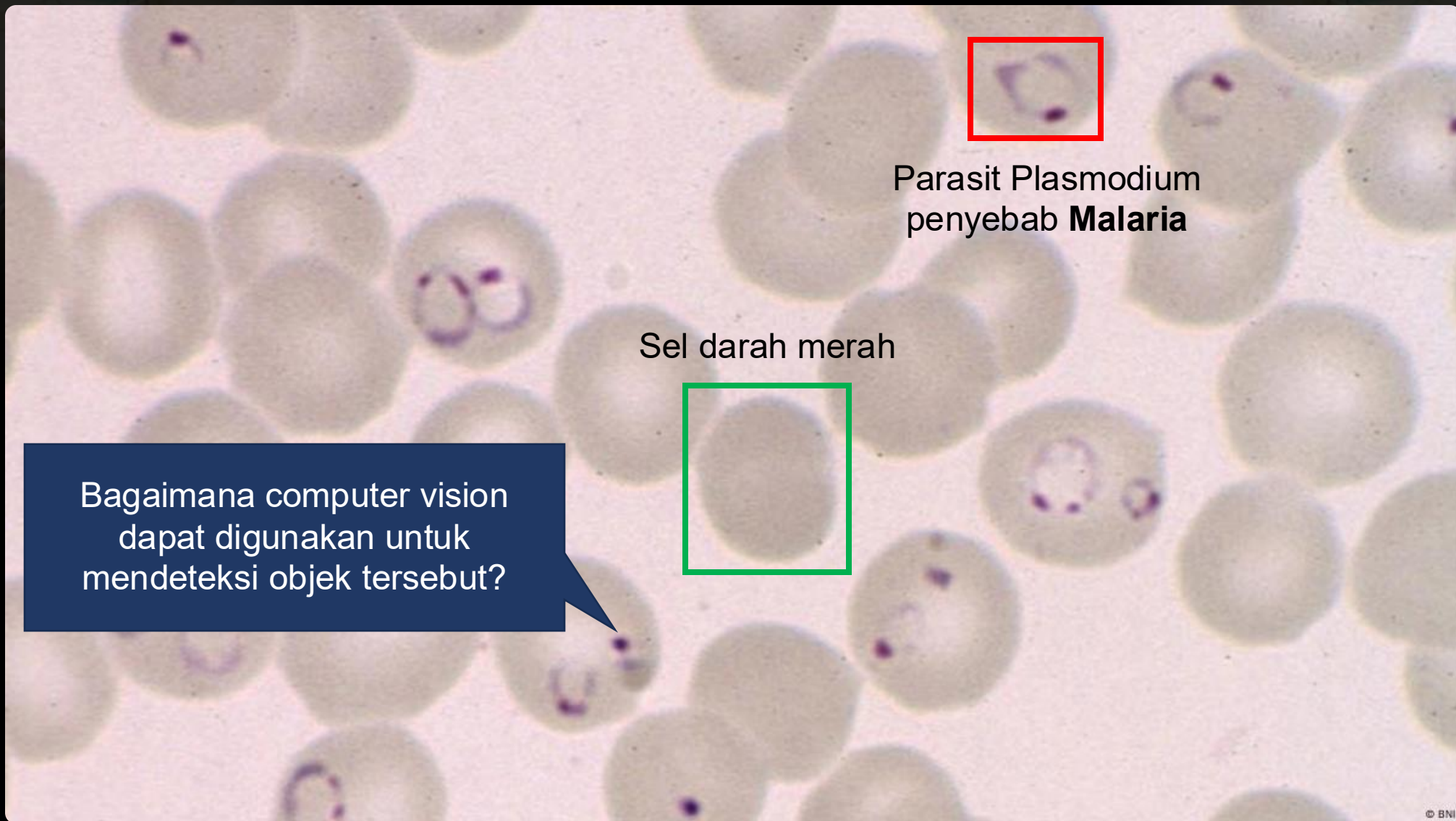
Pertemuan 5 & 6 – Object Detection using Thresholding Method

ALIFIA REVAN PRANANDA

Department of Information Technology
Faculty of Engineering
Universitas Tidar



OBJECT DETECTION



Gambar di atas merupakan contoh citra mikroskopik digital berupa sel darah yang diambil pada saat pemeriksaan Malaria

KONSEP OBJECT DETECTION



Object Detection :

Merupakan salah satu bidang penting dalam Computer Vision yang bertujuan tidak hanya mengenali apa objek yang ada dalam suatu gambar, tetapi juga di mana objek tersebut berada. Hal ini membedakan object detection dari image classification yang hanya menghasilkan label kelas tanpa informasi posisi.



Konsep Dasar :

Pada dasarnya, object detection menghasilkan keluaran berupa *bounding box* yang membatasi area objek, label kelas yang menunjukkan kategori objek (misalnya mobil, manusia, atau hewan), serta nilai *confidence* yang menunjukkan tingkat keyakinan sistem terhadap prediksi tersebut.

Dengan kata lain, *object detection* adalah kombinasi dari dua tugas utama:

- Lokalisasi objek (*object localization*) → menemukan lokasi objek dalam gambar.
- Klasifikasi objek (*object recognition*) → mengenali jenis objek.

KONSEP OBJECT DETECTION

Secara umum, terdapat dua jalur besar dalam pendekatan object detection, yaitu pendekatan klasik berbasis **thresholding** dan pendekatan modern berbasis **machine learning** maupun **deep learning**.

Thresholding merupakan metode sederhana yang bekerja dengan prinsip pemisahan piksel gambar berdasarkan intensitas atau warna tertentu. Prosesnya biasanya diawali dengan **preprocessing**, misalnya konversi gambar ke grayscale atau HSV, serta penerapan filter untuk mengurangi noise. Setelah itu, sebuah **nilai ambang (threshold)** ditentukan untuk memisahkan piksel foreground (objek) dan background.

Jika target adalah objek dengan warna dominan tertentu, seperti deteksi bola merah atau area ungu, maka **thresholding warna** dapat diterapkan dengan memilih rentang nilai tertentu pada ruang warna HSV. Hasil pemisahan ini kemudian diproses untuk menemukan kontur objek, yang dapat dikonversi menjadi bounding box sederhana.

Walaupun metode ini efektif untuk kasus objek yang kontras dengan latar belakang, ia memiliki keterbatasan jika digunakan pada gambar dengan pencahayaan bervariasi atau latar belakang kompleks. Namun, dalam aplikasi industri sederhana, **thresholding** sering digunakan karena cepat, ringan, dan tidak memerlukan data latih.

EVALUATION

Salah satu ukuran paling fundamental adalah Confusion Matrix, yang mengklasifikasikan hasil prediksi ke dalam empat kategori:

- True Positive (objek benar terdeteksi),
- False Positive (terdeteksi padahal tidak ada),
- False Negative (tidak terdeteksi padahal ada), dan
- True Negative (latar belakang benar diabaikan).

	Predicted 	Predicted 
Actual 	4 [TN]	2 [FP]
Actual 	1 [FN]	3 [TP]

Precision is indicated by a blue arrow pointing to the 2 [FP] cell.

Recall / True Positive Rate / Sensitivity is indicated by a yellow arrow pointing to the 3 [TP] cell.

$$\text{Precision} = \frac{3}{3+2} = 0.6 \Rightarrow 60\%$$

$$\text{Recall} = \frac{3}{3+1} = 0.75 \Rightarrow 75\%$$

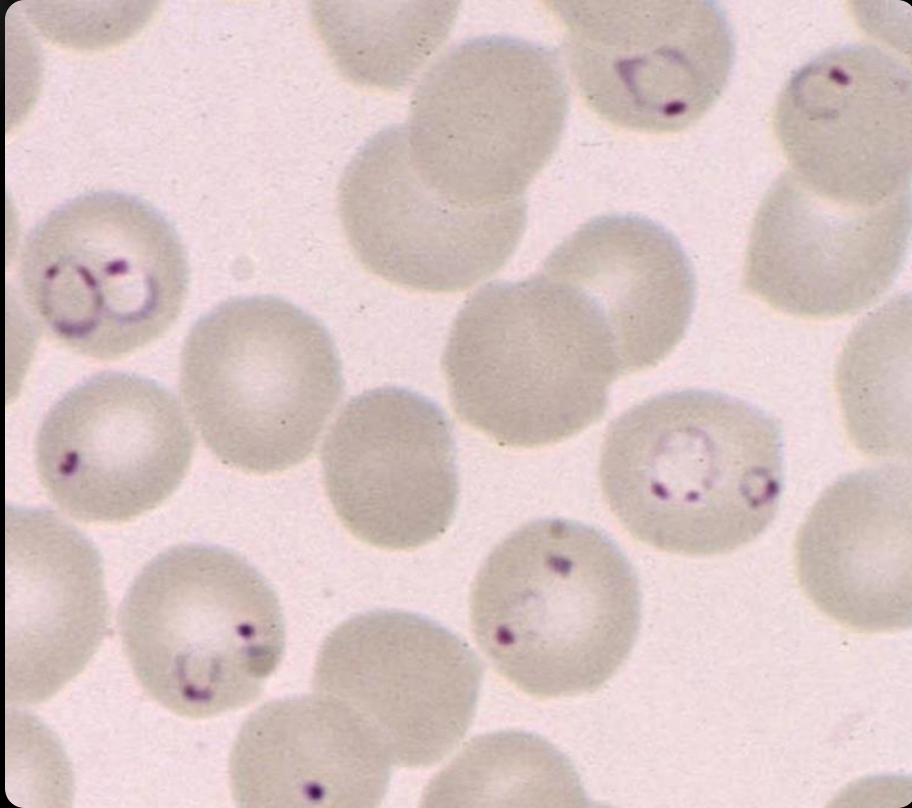
$$\text{F1 Score} = \frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 * 0.6 * 0.75}{0.6 + 0.75} = 0.6666 = 0.67$$

Dari confusion matrix, kita bisa menurunkan ukuran lain seperti:

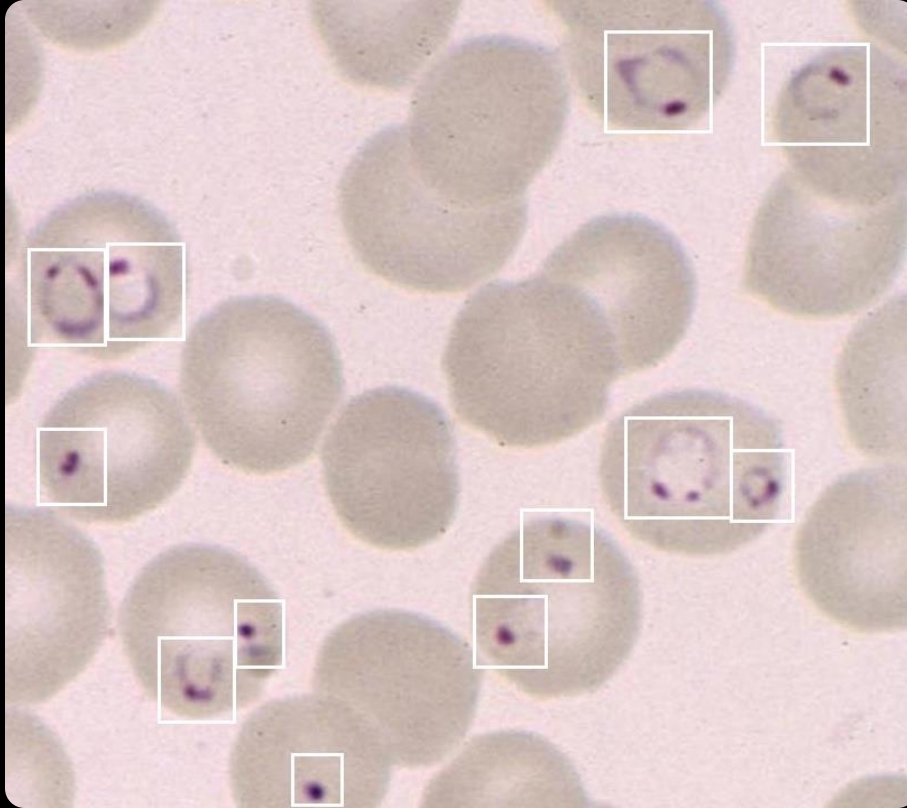
- **Precision**, yaitu seberapa akurat deteksi yang dilakukan model;
- **Recall**, yaitu seberapa banyak objek yang berhasil ditemukan dari keseluruhan objek sebenarnya;

OBJECT DETECTION MENGGUNAKAN THRESHOLDING

Data yang perlu di siapkan dalam proses object detection:



Raw data



Ground truth visual

```
<?xml version="1.0" ?>
<annotation>
  <folder>data_input</folder>
  <filename>ground-truth.png</filename>
  <path>ground-truth.png</path>
  <source>
    <database>Unknown</database>
  </source>
  <size>
    <width>1280</width>
    <height>720</height>
    <depth>3</depth>
  </size>
  <segmented>0</segmented>
  <object>
    <name>cell</name>
    <pose>Unspecified</pose>
    <truncated>0</truncated>
    <difficult>0</difficult>
    <bndbox>
      <xmin>581</xmin>
      <ymin>671</ymin>
      <xmax>628</xmax>
      <ymax>719</ymax>
    </bndbox>
  </object>
```

Ground truth XML

Dalam kasus object detection, ground truth dalam bentuk XML sangat diperlukan untuk melakukan evaluasi

OBJECT DETECTION MENGGUNAKAN THRESHOLDING

```
import cv2
import numpy as np
import xml.etree.ElementTree as ET
import os
from google.colab import drive
import matplotlib.pyplot as plt

# ==== 1. Mount Google Drive ====
drive.mount('/content/drive')

# Path folder data_input di Google Drive
data_folder = "/content/drive/MyDrive/Running/ComputerVision/Week-5-6"

# Nama file (pastikan ada ground_truth.png dan ground_truth.xml di folder tsb)
img_name = "ground-truth.png"
xml_name = "ground-truth.xml"
img_path = os.path.join(data_folder, img_name)
xml_path = os.path.join(data_folder, xml_name)

# ==== 2. Load Image ====
img = cv2.imread(img_path)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# ==== 3. Deteksi warna ungu pakai thresholding (HSV) ====
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Rentang warna ungu (silakan sesuaikan jika kurang pas)
lower_purple = np.array([120, 40, 40])
upper_purple = np.array([160, 255, 255])

mask = cv2.inRange(hsv, lower_purple, upper_purple)
```

```
# Cari kontur hasil threshold
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
pred_bboxes = []
for cnt in contours:
    x, y, w, h = cv2.boundingRect(cnt)
    if w > 5 and h > 5: # filter noise
        pred_bboxes.append((x, y, x+w, y+h))
        cv2.rectangle(img, (x,y), (x+w, y+h), (0,255,0), 2) # hijau: prediksi

# ==== 4. Load Ground Truth dari XML (Pascal VOC) ====
def parse_voc_xml(xml_file):
    tree = ET.parse(xml_file)
    root = tree.getroot()
    bboxes = []
    for obj in root.findall("object"):
        bndbox = obj.find("bndbox")
        xmin = int(bndbox.find("xmin").text)
        ymin = int(bndbox.find("ymin").text)
        xmax = int(bndbox.find("xmax").text)
        ymax = int(bndbox.find("ymax").text)
        bboxes.append((xmin, ymin, xmax, ymax))
    return bboxes

gt_bboxes = parse_voc_xml(xml_path)
```

Nilai threshold harus diperhatikan, berdasarkan warna atau nilai piksel tertentu

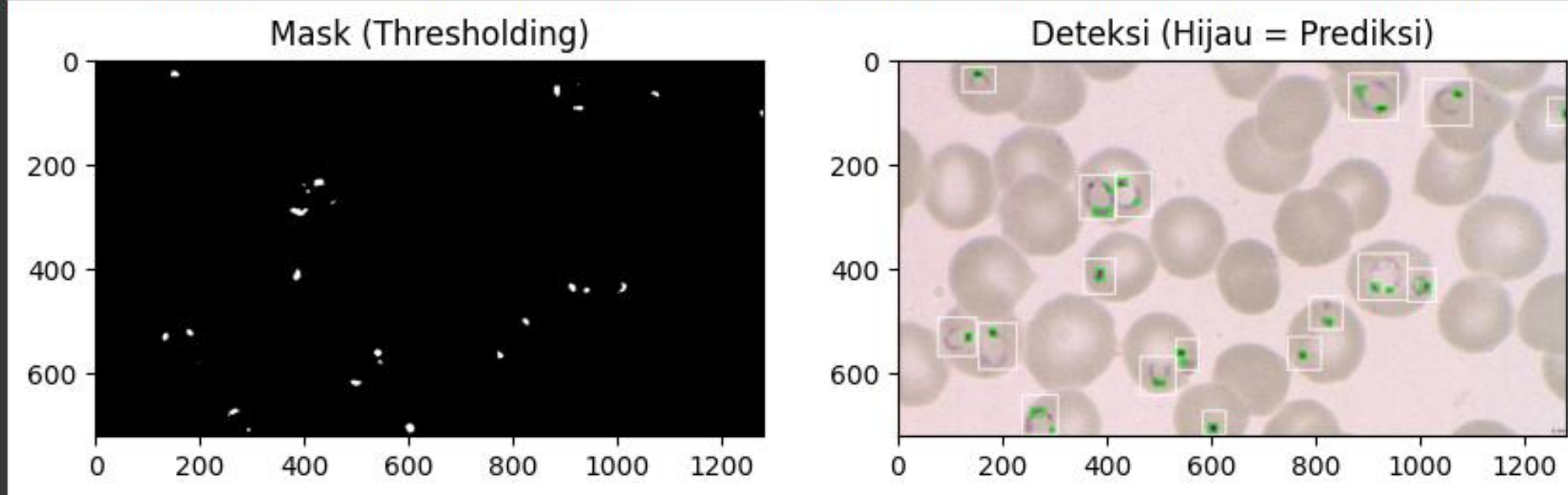
OBJECT DETECTION MENGGUNAKAN THRESHOLDING

```
# ==== 5. Evaluasi Precision & Recall (IoU + Centroid) ====  
def iou(boxA, boxB):  
    xA = max(boxA[0], boxB[0])  
    yA = max(boxA[1], boxB[1])  
    xB = min(boxA[2], boxB[2])  
    yB = min(boxA[3], boxB[3])  
  
    interArea = max(0, xB - xA) * max(0, yB - yA)  
    boxAArea = (boxA[2]-boxA[0]) * (boxA[3]-boxA[1])  
    boxBArea = (boxB[2]-boxB[0]) * (boxB[3]-boxB[1])  
    return interArea / float(boxAArea + boxBArea - interArea + 1e-6)  
  
def is_center_inside(pred, gt):  
    cx = (pred[0] + pred[2]) // 2  
    cy = (pred[1] + pred[3]) // 2  
    return gt[0] <= cx <= gt[2] and gt[1] <= cy <= gt[3]  
  
TP = 0  
FP = 0  
FN = 0
```

```
matched_gt = set()  
iou_thresh = 0.2 # lebih toleran untuk objek kecil  
  
for pred in pred_bboxes:  
    found_match = False  
    for i, gt in enumerate(gt_bboxes):  
        if i in matched_gt:  
            continue  
        if iou(pred, gt) > iou_thresh or is_center_inside(pred, gt):  
            TP += 1  
            matched_gt.add(i)  
            found_match = True  
            break  
    if not found_match:  
        FP += 1  
  
FN = len(gt_bboxes) - TP  
  
precision = TP / (TP + FP + 1e-6)  
recall = TP / (TP + FN + 1e-6)  
f1 = 2 * precision * recall / (precision + recall + 1e-6)
```

OBJECT DETECTION MENGGUNAKAN THRESHOLDING

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

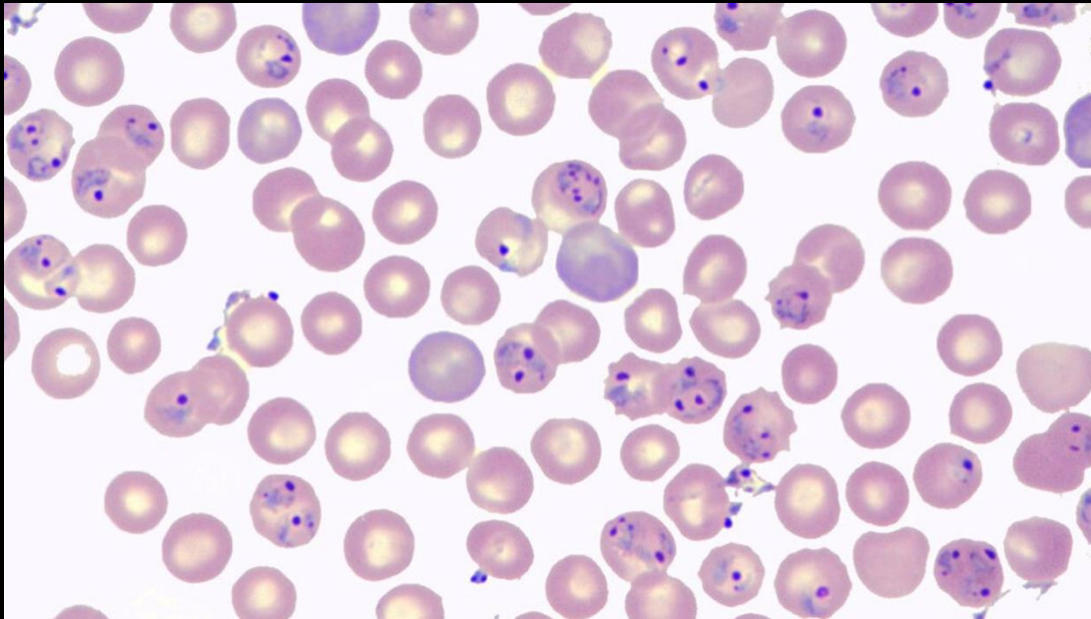


Evaluasi Deteksi:
True Positive: 14
False Positive: 10
False Negative: 0
Precision: 0.583
Recall: 1.000
F1-Score: 0.737

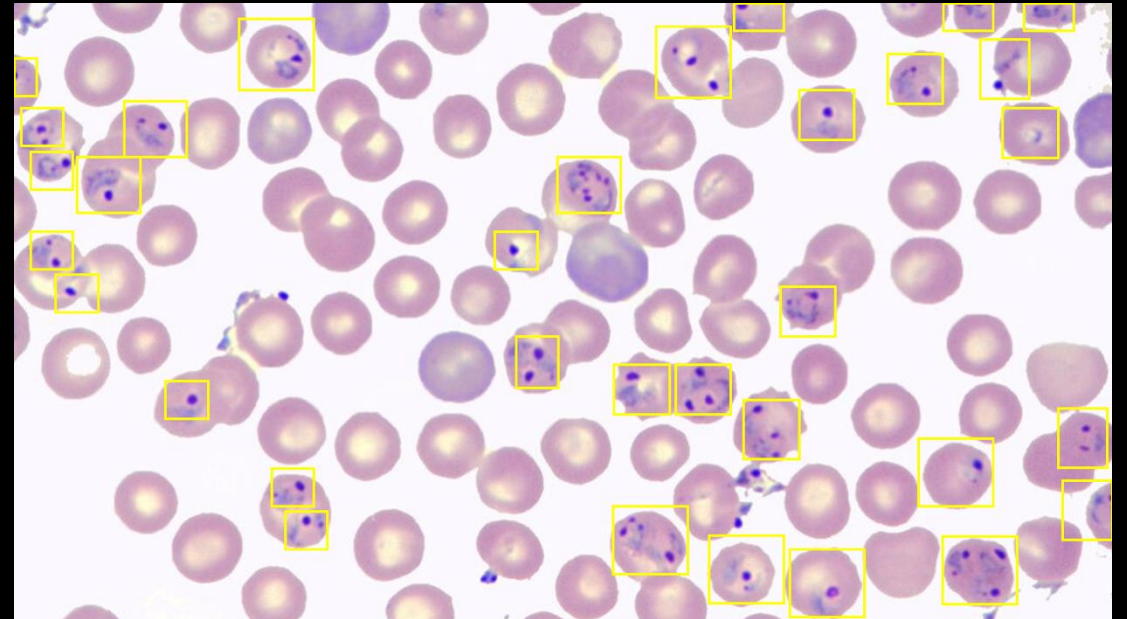
TUGAS

Berdasarkan gambar terlampir, lakukan:

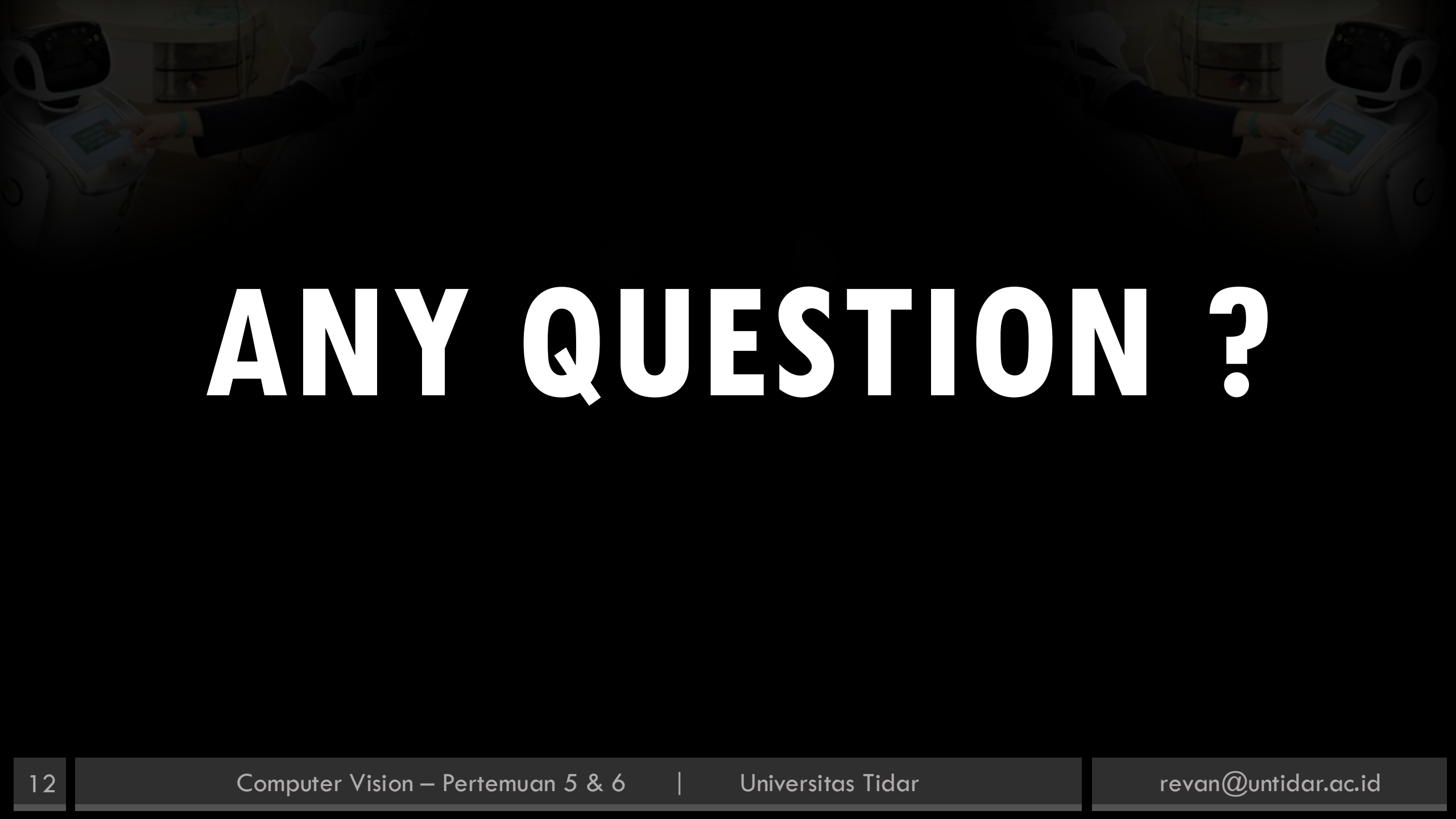
1. Pembuatan ground truth dalam bentuk XML/JS/etc
2. Pembuatan code untuk tujuan object detection menggunakan metode thresholding
3. Evaluasi dalam bentuk visual dan perhitungan precision, recall dan f1-score
4. Lakukan preprocessing seperti filtering atau color transformation apabila diperlukan



Raw data



Ground truth visual



ANY QUESTION ?